PRÁCTICA 1

Gestión de procesos y comunicación entre procesos

Sistemas Operativos · Curso 2026/2027

Francisco Javier Galiano Carratalá
55019705W
Grupo: 40
GitHub: https://github.com/fjgc22/Practicas-Sistemas-Operativos/tree/main/Prac1

Índice

1. Introducción

2. Ejercicio 1.a: malla.c

3. Ejercicio 1.b: ejec.c

4. Ejercicio 2: hacha.c

5. Ejercicio 3: hijos.c

6. Compilación y comprobaciones

7. Conclusiones

1. Introducción

En esta práctica he trabajado con procesos en Unix/Linux y con distintas formas de coordinarlos. Los dos primeros ejercicios se centran en crear procesos y controlar cuándo terminan, usando fork(), wait() y señales. En hacha.c he usado una tubería para pasar los datos de un archivo a un hijo, y en hijos.c he recurrido a memoria compartida para guardar los PID que necesitan consultar los procesos.

También he añadido comprobaciones para los argumentos y para algunas llamadas al sistema. La idea es que, si se introduce un dato incorrecto o falla una operación, el programa lo detecte en lugar de seguir ejecutándose como si nada.

2. Ejercicio 1.a: creación de la malla de procesos (malla.c)

2.1. Objetivo

El programa tiene que crear una estructura con x filas e y columnas. malla crea un hijo por cada columna y cada uno de esos hijos va creando los procesos que faltan hacia abajo. Por eso, los procesos de una misma columna forman una cadena y los de la primera fila dependen directamente de malla. Para revisar que queda así, se puede usar pstree -c.

2.2. Cómo lo he resuelto

Lo primero es comprobar que se han escrito los dos argumentos y que ambos son números positivos. Para comprobar que solo hay dígitos uso soloDigitos(); después convierto las cadenas a enteros con atoi().

Con los argumentos comprobados, malla muestra su PID y crea los hijos de la primera fila con un bucle. Cada hijo entra en bajarColumna(), donde recibe el número de columna y el total de filas. A partir de ahí crea los procesos que le corresponden en esa columna.

Hay que distinguir bien qué hace cada proceso después de fork(). Si devuelve un PID positivo, estamos en el padre: espera al hijo con wait() y sale. Si devuelve cero, estamos en el hijo y continúa la creación de la siguiente fila. Así cada columna sigue su propia cadena y no se crean columnas de más.

Al llegar a la última fila ya no se crean más procesos. Los dejo vivos durante 20 segundos (ESPERA) para poder consultar el árbol con pstree -c. Para eso uso alarm() y pause(), junto con el manejador despertar() para SIGALRM, que permite que pause() termine cuando llega la señal. Mientras tanto, malla espera a los hijos de la primera fila.

2.3. Llamadas y funciones utilizadas

- fork(): crea los procesos hijos de la primera fila y los siguientes procesos de cada columna.
- wait(): permite que cada padre espere a su hijo y que malla espere a los procesos de la primera fila.
- getpid() y getppid(): obtienen el PID del proceso actual y el PID de su padre.
- signal(), alarm() y pause(): mantienen temporalmente vivos los procesos de la última fila para facilitar la comprobación del árbol.
- write(): lo uso para mostrar los mensajes desde imprimir(); el texto lo preparo antes con sprintf(). Preferí write() a printf() porque printf() puede dejar texto en un búfer; después de un fork(), ese búfer se copia y los mensajes pueden repetirse si la salida está redirigida a un archivo.

2.4. Cómo comprobarlo

Lo he compilado y ejecutado con estos comandos:

```
gcc -Wall -o malla malla.c

./malla 3 4

pstree -c <PID_de_malla>
```

He ejecutado el programa con x = 3 e y = 4. Mientras los procesos siguen activos, desde otra terminal se puede consultar el árbol con pstree, usando el PID que muestra malla:

```
$ ./malla 3 4
Soy malla: mi pid es 229
Soy p11: mi pid es 230. Mi padre es 229
Soy p21: mi pid es 232. Mi padre es 230
Soy p31: mi pid es 233. Mi padre es 232
Soy p12: mi pid es 231. Mi padre es 229
Soy p13: mi pid es 234. Mi padre es 229
Soy p22: mi pid es 236. Mi padre es 231
Soy p32: mi pid es 238. Mi padre es 236
Soy p23: mi pid es 237. Mi padre es 234
Soy p14: mi pid es 239. Mi padre es 229
Soy p33: mi pid es 240. Mi padre es 237
Soy p24: mi pid es 241. Mi padre es 239
Soy p34: mi pid es 242. Mi padre es 241
Soy malla (229) y muero
```

```
$ pstree -c 229
malla-+-malla---malla---malla
      |-malla---malla---malla
      |-malla---malla---malla
      `-malla---malla---malla
```

La salida de pstree muestra cuatro cadenas, cada una con tres procesos, y todas dependen de malla. El mensaje final de malla aparece cuando han pasado los 20 segundos y las columnas han terminado. Los mensajes de creación no siempre salen en el mismo orden, ya que depende de cómo el sistema vaya ejecutando los procesos.

3. Ejercicio 1.b: coordinación con señales (ejec.c)

3.1. Objetivo

El programa recibe un número de segundos y crea el árbol ejec → A → B → X, Y, Z. Z espera ese tiempo sin usar sleep(); después avisa a A mediante una señal para que A ejecute pstree. Una vez hecho esto, los procesos deben terminar en el orden indicado y cada padre tiene que esperar a sus hijos.

3.2. Creación del árbol

Primero compruebo que solo se haya pasado un argumento y que sea un número entero positivo, usando también soloDigitos(). Guardo el tiempo de espera y obtengo el PID de ejec. Después creo A con fork(); A crea B mediante crearB(), y B crea X, Y y Z llamando a nuevaHoja() con la letra correspondiente. Cada proceso muestra su PID y los de sus antecesores.

Los manejadores de señales necesitan saber a qué proceso deben enviar la siguiente señal, así que guardo esos PID en variables globales. Como el manejador solo recibe el número de señal, no podría

obtenerlos directamente de sus parámetros. Instalo los manejadores antes de crear los procesos que van a enviar las señales.

3.3. Secuencia de señales

1. Z programa una alarma con alarm(espera) y se queda esperando con pause(). Cuando llega SIGALRM, se ejecuta avisarA(), que envía SIGUSR1 a A.
2. Al recibir SIGUSR1, A ejecuta lanzarPstree(). El manejador crea un hijo con fork() y ese hijo ejecuta pstree -c con execlp(). De este modo, el comando se ejecuta en el hijo y A conserva su propio programa.
3. Cuando termina el hijo que ejecuta pstree, A le envía SIGUSR2 a ejec para avisar de que ya puede comenzar el cierre.
4. Al recibir SIGUSR2, ejec ejecuta iniciarCierre() y envía la misma señal a A. A se la pasa a B mediante el manejador cierreA().
5. B recibe la señal y empieza a cerrar sus hijos en el orden Z, Y y X. Después de enviar la señal de cierre a cada uno, espera con wait() antes de continuar con el siguiente. Cada hoja ejecuta cierreHoja(), muestra su mensaje y termina con exit(0).
6. Cuando ya han terminado X, Y y Z, B puede terminar. A espera a B y ejec espera a A. Uso while (wait(NULL) != pid) porque una señal puede interrumpir wait() antes de que el hijo haya acabado; el bucle permite seguir esperando hasta recoger el PID que toca.

3.4. Llamadas y funciones utilizadas

- fork(): crea A, B, X, Y, Z y el hijo auxiliar que ejecuta pstree.
- getpid() y getppid(): obtienen el PID de cada proceso y el de su padre para los mensajes.
- signal(): registra los manejadores de SIGALRM, SIGUSR1 y SIGUSR2.
- alarm(): programa la señal de Z sin utilizar sleep().
- kill(): envía señales a los procesos que deben realizar la siguiente acción.
- pause(): mantiene X, Y y Z esperando hasta recibir la señal de cierre.
- execlp(): sustituye el proceso auxiliar por el comando pstree -c.
- wait(): espera a los hijos y ayuda a garantizar que los padres no terminen antes que ellos.
- write(): muestra los mensajes sin depender del búfer de printf(). También permite escribir directamente desde los manejadores de señales.

3.5. Cómo comprobarlo

```
gcc -Wall -o ejec ejec.c

./ejec 15
```

Al ejecutar ./ejec 15, primero aparecen los PID de los procesos. Al cabo de 15 segundos se muestra el árbol con pstree y después se imprimen los mensajes de terminación en este orden: Z, Y, X, B, A y ejec.

```
$ ./ejec 15
Soy el proceso ejec: mi pid es 260
Soy el proceso A: mi pid es 261. Mi padre es 260
Soy el proceso B: mi pid es 262. Mi padre es 261. Mi abuelo es 260
Soy el proceso Y: mi pid es 264. Mi padre es 262. Mi abuelo es 261. Mi bisabuelo es 260
Soy el proceso X: mi pid es 263. Mi padre es 262. Mi abuelo es 261. Mi bisabuelo es 260
Soy el proceso Z: mi pid es 265. Mi padre es 262. Mi abuelo es 261. Mi bisabuelo es 260
ejec---ejec-+-ejec-+-ejec
            |      |-ejec
            |      `-ejec
            `-pstree
Soy Z (265) y muero
Soy Y (264) y muero
Soy X (263) y muero
```

```
Soy B (262) y muero
Soy A (261) y muero
Soy ejec (260) y muero
```

En pstree todos aparecen con el nombre ejec porque se han creado con fork() y siguen ejecutando el mismo programa. También se ve el proceso auxiliar que A crea para lanzar pstree. Los mensajes iniciales de X e Y pueden intercambiarse de orden, pero la terminación sí sigue la secuencia prevista.

4. Ejercicio 2: división de archivos mediante tuberías (hacha.c)

4.1. Objetivo

hacha recibe el nombre de un archivo y el tamaño, en bytes, que debe tener cada fragmento. Los archivos generados se llaman, por ejemplo, archivo.h00, archivo.h01, etc. El padre lee el original y pasa los datos por una tubería a un hijo, que crea el fragmento y escribe en él. En este ejercicio no se pueden usar funciones como printf() o scanf(), así que la entrada y salida se hacen con llamadas al sistema.

4.2. Cómo lo he resuelto

Primero reviso que estén los dos argumentos, que el tamaño sea positivo y que el nombre del archivo no supere la longitud permitida. Abro el original con open() y uso lseek() para conocer su tamaño, antes de volver al principio. Si no se puede abrir o está vacío, muestro el error y termino.

Con el tamaño del archivo y el tamaño pedido calculo cuántos fragmentos necesito. Redondeo hacia arriba para no perder los datos que queden en el último fragmento. Después, para cada fragmento, creo una tubería con pipe() y un hijo con fork().

El hijo entra en hijo(). Cierra el extremo de escritura de la tubería y el descriptor del original, que no va a utilizar. Luego nombreTrozo() construye el nombre del archivo: añade .h y el número con al menos dos cifras (h00, h01... h99; a partir de ahí, h100, h101...). Lo hago cifra a cifra porque no puedo usar sprintf(). Después crea el archivo con creat() y copia los datos que recibe de la tubería, en bloques de hasta 4096 bytes.

Por su parte, el padre cierra el extremo de lectura y manda los datos del fragmento con mandarTrozo(), que indica si la operación ha salido bien. Se envían bloques, así que no hace falta cargar todo el archivo en memoria. Al terminar, el padre cierra el extremo de escritura; entonces el hijo detecta el final cuando read() devuelve cero. El padre espera al hijo con wait(&status) antes de pasar al fragmento siguiente, por lo que se procesan de uno en uno. Si falla el envío o el hijo no termina correctamente, se muestra un error.

Ignoro SIGPIPE para poder detectar con write() el caso en que el hijo deja de leer, en vez de que el padre termine directamente por la señal. Los errores se escriben en la salida de errores, descriptor 2, mediante la función fallo() y write().

4.3. Llamadas y funciones utilizadas

- open(): abre el archivo original en modo lectura.
- close(): cierra el archivo original, los fragmentos y los extremos de cada tubería que no se usan.
- lseek(): obtiene el tamaño del archivo y vuelve a colocar la posición de lectura al principio.
- pipe(): crea el canal de comunicación entre padre e hijo.
- fork(): crea un hijo por cada fragmento.
- read() y write(): transfieren los datos entre el archivo, la tubería y el archivo de destino.
- creat(): crea el archivo de cada fragmento.

- wait(): espera al hijo actual antes de continuar con el siguiente fragmento y permite saber cómo ha terminado.
- signal(SIGPIPE, SIG_IGN): permite tratar el fallo de escritura como un error controlado.

4.4. Cómo comprobarlo

```
gcc -Wall -o hacha hacha.c

./hacha archivo_prueba.bin 50000

ls -l archivo_prueba.bin*
```

Para probarlo utilicé un archivo de 130000 bytes y fragmentos de 50000. Como el tamaño total no es múltiplo de 50000, el último fragmento debe ser más pequeño. Después junté los tres archivos en orden y comparé el resultado con el original:

```
$ ./hacha archivo_prueba.bin 50000
$ ls -l archivo_prueba.bin*
-rw-r--r-- 1 root root 130000 Oct 10 19:13 archivo_prueba.bin
-rw-r--r-- 1 root root  50000 Oct 10 19:13 archivo_prueba.bin.h00
-rw-r--r-- 1 root root  50000 Oct 10 19:13 archivo_prueba.bin.h01
-rw-r--r-- 1 root root  30000 Oct 10 19:13 archivo_prueba.bin.h02
$ cat archivo_prueba.bin.h00 archivo_prueba.bin.h01 archivo_prueba.bin.h02 > reconstruido.bin
$ cmp archivo_prueba.bin reconstruido.bin
$
```

Se han creado dos fragmentos de 50000 bytes y uno de 30000. cmp no muestra ninguna diferencia, así que el archivo reconstruido coincide con el original.

También comprobé el caso en que el hijo no puede crear el archivo de salida. Para provocarlo, se puede hacer que ya exista un directorio con el nombre que necesita el fragmento. El padre detecta el fallo y termina mostrando un error:

```
$ ./hacha f.bin 100
Error al crear el archivo del trozo
Error: no se ha podido crear uno de los trozos
```

5. Ejercicio 3: árbol de procesos y memoria compartida (hijos.c)

5.1. Objetivo

El programa recibe x e y. Crea primero una cadena de x procesos y, desde el último de esa cadena, crea y subhijos. Al final, el superpadre debe mostrar los PID de esos subhijos y cada subhijo debe mostrar los PID de los procesos que tiene como antecesores.

5.2. Cómo lo he resuelto

Antes de crear los procesos compruebo los dos argumentos y que sean positivos. Después reservo memoria compartida para x + y enteros mediante creaComp(), siguiendo el ejemplo del guion. Uso shmget() para crear el segmento y shmat() para conectarlo al proceso. Inicializo la tabla lista a cero. Todo esto lo hago antes de hacer fork(), de modo que todos los procesos puedan acceder al mismo segmento.

La cadena se crea con un bucle y llamadas a fork(). Cada hijo guarda su PID en la posición que le toca de la memoria compartida y sigue con la siguiente iteración. El padre sale del bucle, así que la estructura queda como una cadena y no como varias ramas.

Cuando llega al último proceso de la cadena, este llama a crearSubhijos() y crea los y subhijos. Cada uno guarda su PID en la parte final de la tabla y muestra los PID de los procesos que forman la cadena de antecesores. El último proceso espera a que terminen todos sus subhijos.

Uso memoria compartida porque el superpadre necesita leer los PID de procesos que están varias generaciones por debajo. Una variable normal no serviría, ya que después de fork() cada proceso tiene su propio espacio de memoria. En cambio, todos acceden al mismo segmento y el superpadre puede consultar los valores que han escrito los subhijos.

Cada padre espera a su hijo con wait(). Así, cuando el superpadre muestra los PID finales, los procesos ya han terminado su parte. Si falla un fork(), el error se propaga hacia arriba con exit(status != 0) y el superpadre lo comunica. Al acabar, se desconecta de la memoria con shmdt() y elimina el segmento con shmctl(..., IPC_RMID, ...) dentro de borraComp().

5.3. Llamadas y funciones utilizadas

- shmget(): reserva el segmento de memoria compartida para x + y enteros.
- shmat(): conecta el segmento al espacio de memoria del proceso.
- shmdt(): desvincula el segmento cuando ya no se necesita.
- shmctl() con IPC_RMID: elimina el segmento de memoria compartida del sistema.
- fork(): crea la cadena de procesos y los subhijos finales.
- wait(): coordina la terminación y permite que la información esté escrita antes de mostrarla desde el superpadre.
- getpid(): obtiene el PID que cada proceso almacena en la tabla compartida.

5.4. Cómo comprobarlo

```
gcc -Wall -o hijos hijos.c
```

```
./hijos 3 4
```

Con x = 3 e y = 4, la salida muestra una cadena de tres procesos y cuatro subhijos creados desde el último de ellos:

```
$ ./hijos 3 4
Soy el subhijo 252, mis padres son: 249, 250, 251
Soy el subhijo 253, mis padres son: 249, 250, 251
Soy el subhijo 254, mis padres son: 249, 250, 251
Soy el subhijo 255, mis padres son: 249, 250, 251
Soy el superpadre (248) : mis hijos finales son: 252, 253, 254, 255
$ ipcs -m
------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status
```

Los cuatro subhijos muestran los mismos tres PID de la cadena (249, 250 y 251). Además, los PID que imprime el superpadre coinciden con los de esos subhijos. La consulta con ipcs -m no muestra ningún segmento al terminar, lo que indica que se ha eliminado la memoria compartida.

6. Compilación y comprobaciones

He compilado cada programa con gcc y la opción -Wall para que el compilador avise de posibles problemas. En las compilaciones que aparecen en esta memoria no se muestran avisos. Los comandos utilizados son:

```
gcc -Wall -o malla malla.c
```

```
gcc -Wall -o ejec ejec.c

gcc -Wall -o hacha hacha.c

gcc -Wall -o hijos hijos.c
```

También he probado algunos argumentos incorrectos: omitir argumentos, escribir letras donde se espera un número o usar cero. En los ejemplos siguientes se ve que los programas muestran el mensaje correspondiente y terminan:

```
$ ./malla
Uso: ./malla <filas> <columnas>
$ ./malla 0 3
Error: las filas y las columnas tienen que ser mayores que 0
$ ./malla 2x 3
Error: las filas y las columnas tienen que ser numeros
$ ./ejec 5abc
Error: los segundos tienen que ser un numero mayor que 0
$ ./hacha archivo_prueba.bin 0
Error: el tamano tiene que ser un numero mayor que 0
$ ./hacha noexiste.bin 100
Error al abrir el archivo
$ ./hacha vacio.bin 10
Error: el archivo esta vacio
$ ./hijos 0 2
Error: x e y tienen que ser numeros mayores que 0
```

7. Conclusiones

Con estos cuatro ejercicios he practicado varias formas de trabajar con procesos. En malla.c he creado procesos organizados en columnas; en ejec.c he usado señales y alarmas para controlar una acción y cerrar los procesos en un orden concreto. En hacha.c he pasado datos mediante una tubería para dividir un archivo, y en hijos.c he utilizado memoria compartida para que el superpadre pueda consultar los PID de los subhijos.

Lo que más se repite en los ejercicios es la necesidad de controlar bien la vida de cada proceso: no basta con crearlo, también hay que saber cuándo espera, cuándo termina y qué recursos tiene que cerrar. Para eso han sido importantes fork() y wait(), además de las señales, las tuberías y las funciones de memoria compartida.